Convolutional Neural Networks for NLP

CS 115B – Spring 2026

James Pustejovsky
March 10, 2026

Course Context

Neural NLP Sequence:

1. **Today: CNNs** — Local pattern detection
2. Next: RNNs — Sequential processing
3. Then: Attention — Flexible context
4. Finally: LLMs — Large-scale pre-training

Why CNNs?

- Efficient feature extraction
- Parallelizable (unlike RNNs)
- Good for local patterns (n-grams)
- Still used in modern architectures

Roadmap

Part I: CNN Basics from Vision (25 min)

- What is convolution?
- Filters and feature detection
- Pooling operations

Part II: Adapting CNNs to Text (20 min)

- Text as sequence of embeddings
- 1D convolution over sequences
- Why CNNs make sense for language

Part III: CNNs for Sentence Classification (35 min)

- Kim (2014): Convolutional Neural Networks for Sentence Classification
- Architecture details and worked examples
- Multiple filter sizes and max-pooling

Part IV: Advanced Topics (10 min)

What is a Convolution?

Intuition: Slide a small pattern detector (filter) over input

Mathematical definition (1D):

$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m] \cdot g[n-m]$$

For discrete signals with finite support:

$$(f * g)[n] = \sum_{m=0}^{k-1} f[m] \cdot g[n-m]$$

In neural networks:

- f is the **filter/kernel** (learned parameters)
- g is the **input** (data)
- Output is **feature map** (activations)

Visual Example: Edge Detection

Image: 2D grid of pixel values
Filter: 3×3 kernel to detect vertical edges

Input

10	10	10	90	90
10	10	10	90	90
10	10	10	90	90
10	10	10	90	90

Filter

-1	0	1
-1	0	1
-1	0	1

Output

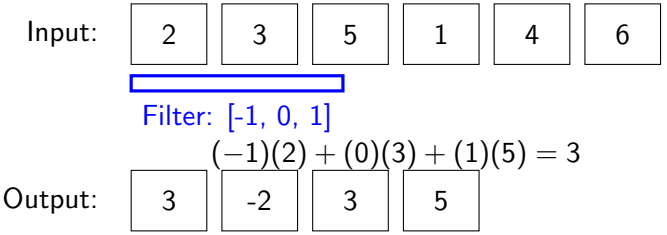
0	0	240	0
0	0	240	0

High activation where vertical edge exists!

- Filter detects transition from dark (10) to bright (90)
- Maximum response at boundary

Convolution Operation: Step by Step

1D Example: Detect pattern in sequence [2, 3, 5, 1, 4, 6]
Filter: [-1, 0, 1] (detects increasing patterns)



Stride: How much we shift filter (typically 1)
Padding: Add zeros at boundaries (to maintain size)

Key CNN Concepts

1. **Local connectivity**:
   - Each output unit sees only small region of input
   - Filter size: 3×3, 5×5, etc.
   - Reduces parameters dramatically vs. fully connected
2. **Parameter sharing**:
   - Same filter used across entire input
   - Learn one pattern detector, apply everywhere
   - E.g., 3×3 filter $= 9$ parameters for entire image
3. **Translation invariance**:
   - Detect pattern regardless of position
   - Edge detector works at any location
   - Important for both images and text!

Multiple Filters and Channels

Multiple filters detect different patterns:

- Filter 1: Vertical edges
- Filter 2: Horizontal edges
- Filter 3: Diagonal patterns
- Filter 4: Corners

Output: Multiple feature maps (one per filter)

- Input: $H \times W \times C_{\text{in}}$ (height $\times$ width $\times$ channels)
- Filters: $k \times k \times C_{\text{in}} \times C_{\text{out}}$
- Output: $H' \times W' \times C_{\text{out}}$

Stacking layers:

- Early layers: Simple patterns (edges)
- Middle layers: Parts (eyes, noses in vision)
- Deep layers: Complex objects (faces, cars)

Activation Functions

After convolution, apply non-linearity:

ReLU (most common):

$$f(x) = \max(0, x)$$

- Simple, fast to compute
- Helps with gradient flow
- Introduces non-linearity (enables learning complex functions)

Full operation:

$$\mathbf{h} = \text{ReLU}(\mathbf{W} * \mathbf{x} + \mathbf{b})$$

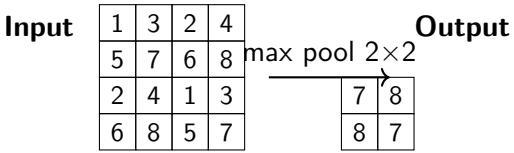
where $*$ is convolution, $\mathbf{W}$ is filter, $\mathbf{b}$ is bias

Pooling Operations

Goal: Downsample feature maps, reduce dimensionality

Max pooling (most common):

- Take maximum value in each region
- Example: 2×2 max pooling with stride 2

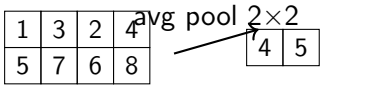


Why pooling?

- Reduces computation
- Provides invariance to small translations
- Captures "most important" features

Average Pooling

Alternative: Take average instead of maximum



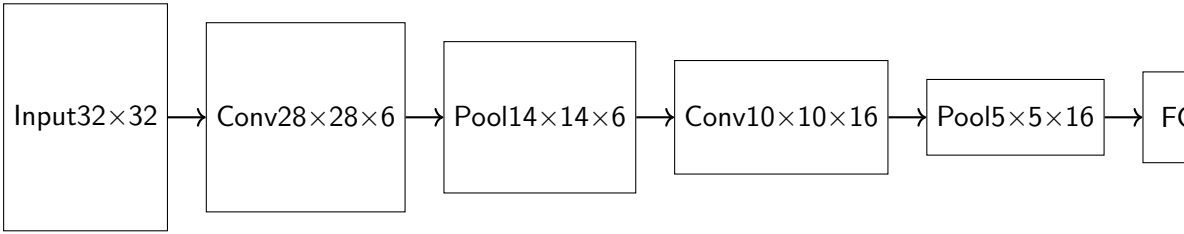
$(1 + 3 + 5 + 7)/4 = 4$, $(2 + 4 + 6 + 8)/4 = 5$

Max vs. Average:

- Max: Emphasizes strongest activations
- Average: Considers all activations equally
- Max is more common in practice

CNN Architecture Example (Vision)

Classic architecture: LeNet-5 (LeCun et al. 1998)



Pattern: Conv → ReLU → Pool → Conv → ReLU → Pool → FC → Output

Key ideas:

- Spatial dimensions decrease (32→14→5)
- Depth (channels) increases (1→6→16)
- Finally: Flatten and use fully connected layers

Why CNNs for Text?

Text seems sequential, not spatial...

But: Local patterns matter!

- N-grams: "not good", "very important", "ice cream"
- Phrases: "New York", "machine learning"
- Syntax: "did not", "would have"

CNNs can detect these patterns:

- Filter size 2 $\rightarrow$ bigrams
- Filter size 3 $\rightarrow$ trigrams
- Filter size 4+ $\rightarrow$ longer phrases

Advantages for text:

- Parallelizable (unlike RNNs)
- Fast training and inference
- Good at capturing local dependencies

Text as Matrix: Word Embeddings

Represent sentence as matrix:

Sentence: "I love this movie"
Word embeddings (e.g., 50-dimensional):

- "I" $\rightarrow \mathbf{e}_1 \in \mathbb{R}^{50}$
- "love" $\rightarrow \mathbf{e}_2 \in \mathbb{R}^{50}$
- "this" $\rightarrow \mathbf{e}_3 \in \mathbb{R}^{50}$
- "movie" $\rightarrow \mathbf{e}_4 \in \mathbb{R}^{50}$

Sentence matrix: $\mathbf{S} \in \mathbb{R}^{4 \times 50}$

- Rows: Words (sequence length n)
- Columns: Embedding dimensions (d)
- Shape: $n \times d$

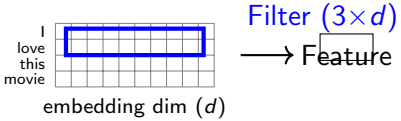
Think of this as an "image" where each word is a row of pixels

1D Convolution Over Text

Filter: Height = window size, Width = embedding dimension

Example: Filter size 3 (trigrams)

- Filter shape: $3 \times d$ (covers 3 words, all dimensions)
- Slide vertically down the sentence
- At each position: compute dot product + bias $\rightarrow$ activation



Output: Feature map capturing presence of trigram patterns

Convolution Details

Input: Sentence matrix $\mathbf{S} \in \mathbb{R}^{n \times d}$

- n = sentence length
- d = embedding dimension

Filter: $\mathbf{W} \in \mathbb{R}^{h \times d}$

- h = filter height (e.g., 3 for trigrams)
- d = embedding dimension (same as input)

Convolution at position i:

$$c_i = \text{ReLU}(\sum_{j=0}^{h-1} \sum_{k=0}^{d-1} \mathbf{W}_{j,k} \cdot \mathbf{S}_{i+j,k} + b)$$

Feature map: $\mathbf{c} = [c_1, c_2, \ldots, c_{n-h+1}]$

- Length: $n - h + 1$ (with no padding)
- One value for each valid filter position

Multiple Filters

Use many filters to detect different patterns:

With m filters of size $h \times d$:

- Each filter produces one feature map: $\mathbf{c}^{(1)}, \mathbf{c}^{(2)}, \ldots, \mathbf{c}^{(m)}$
- Each feature map has length $n - h + 1$

Interpretation:

- Filter 1: Detects "not [adjective]" patterns (negation)
- Filter 2: Detects "very [adjective]" (intensification)
- Filter 3: Detects person names (capitalization patterns)
- ...

Filters learn these patterns automatically during training!

Max-Over-Time Pooling

Problem: Variable-length sentences $\rightarrow$ variable-length feature maps
Solution: Max-over-time pooling

For each feature map $\mathbf{c} = [c_1, c_2, \ldots, c_{n-h+1}]$:

$$\tilde{c} = \max(\mathbf{c}) = \max\{c_1, c_2, \ldots, c_{n-h+1}\}$$

Result: Single number (maximum activation) per filter

Interpretation:

- "Did this pattern appear anywhere in the sentence?"
- Captures most important feature regardless of position
- Creates fixed-size representation from variable-length input

With m filters $\rightarrow$ m-dimensional vector for entire sentence

The Influential Paper

Kim (2014): "Convolutional Neural Networks for Sentence Classification"

Task: Sentence-level classification

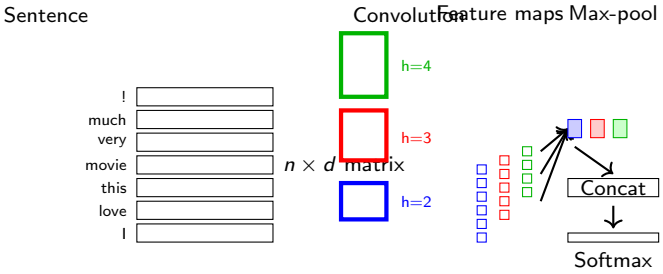
- Sentiment analysis: positive/negative movie reviews
- Question type classification
- Subjectivity detection

Key contributions:

- Simple, effective architecture
- Multiple filter sizes (different n-gram lengths)
- Pre-trained word embeddings (word2vec)
- Various training strategies

Impact: Over 10,000 citations, standard baseline for text classification

Kim CNN Architecture



Key: Multiple filter sizes capture different n-gram lengths!

Architecture Details

Input: Sentence with n words, embedding dimension d

- Sentence matrix: $\mathbf{S} \in \mathbb{R}^{n \times d}$

Convolutional layer: Multiple filter sizes

- Filter sizes: $h \in \{3, 4, 5\}$ (trigrams, 4-grams, 5-grams)
- 100 filters of each size $\rightarrow$ 300 filters total
- Each filter: $h \times d$ parameters

Pooling: Max-over-time pooling

- One value per filter
- Output: 300-dimensional vector

Fully connected + Softmax:

- FC layer with dropout (0.5)
- Softmax over classes

Worked Example: Sentiment Classification

Sentence: "I love this movie"
Task: Binary sentiment (positive/negative)

Step 1: Embeddings

- Each word $\rightarrow$ 300-dim vector (word2vec)
- Matrix: 4×300

Step 2: Convolution with filter size 2

- Filter covers "I love" $\rightarrow$ compute activation c_1
- Slide to "love this" $\rightarrow$ activation c_2
- Slide to "this movie" $\rightarrow$ activation c_3
- Feature map: $[c_1, c_2, c_3]$

Suppose: $c_1 = 0.3$, $c_2 = 0.8$, $c_3 = 0.1$
Max pool: $\max(0.3, 0.8, 0.1) = 0.8$

Worked Example (continued)

Step 3: Multiple filters of size 2

100 filters of size 2, each produces one value after max-pooling:

$$[\text{filter}_1 : 0.8, \text{filter}_2 : 0.3, \ldots, \text{filter}_{100} : 0.5]$$

Step 4: Repeat for filter sizes 3, 4

Filter size 3:

- "I love this" $\rightarrow c_1$
- "love this movie" $\rightarrow c_2$
- Max pool each of 100 filters

Filter size 4:

- "I love this movie" $\rightarrow c_1$ (only one position)
- Max pool each of 100 filters

Step 5: Concatenate

$[100 \text{ from size } 2] + [100 \text{ from size } 3] + [100 \text{ from size } 4] = 300\text{-dim vector}$

Worked Example (continued)

Step 6: Fully connected layer
300-dim vector $\rightarrow$ Dropout (0.5) $\rightarrow$ Linear $\rightarrow$ Softmax

$$\mathbf{z} = \mathbf{W} \cdot \text{dropout}(\mathbf{h}) + \mathbf{b}$$

$$P(\text{pos}) = \frac{\exp(z_{\text{pos}})}{\exp(z_{\text{pos}}) + \exp(z_{\text{neg}})}$$

Example output:

- $P(\text{positive}) = 0.92$
- $P(\text{negative}) = 0.08$

Prediction: Positive

The filter that detected "love this" had high activation, contributing to positive classification!

What Do Filters Learn?

Example learned patterns (from Kim's experiments):

Filter detecting negation (size 2):

- High activation for: "not good", "not great", "not worth"
- This filter has negative weight $\rightarrow$ reduces positive score

Filter detecting intensification (size 2):

- High activation for: "very good", "highly recommend", "extremely well"
- Positive weight $\rightarrow$ increases positive score

Filter detecting superlatives (size 3):

- "one of the best", "one of the worst"
- Context-dependent weighting

Filters automatically discover linguistically meaningful patterns!

Training Strategies

Kim explored several variants:

1. **CNN-rand**: Random initialization
   - Initialize word embeddings randomly
   - Learn from scratch
2. **CNN-static**: Pre-trained, frozen
   - Use word2vec embeddings
   - Keep fixed during training
3. **CNN-non-static**: Pre-trained, fine-tuned
   - Initialize with word2vec
   - Update during training
   - Best performance!
4. **CNN-multichannel**: Both static and non-static
   - Two sets of embeddings
   - One static, one fine-tuned

Results on Sentiment Analysis

Dataset: Movie Review (MR) - binary sentiment

Model	**Accuracy**
Naive Bayes (baseline)	79.4%
SVM with hand-crafted features	79.4%
CNN-rand	76.1%
CNN-static	81.0%
CNN-non-static	**81.5%**
CNN-multichannel	81.1%

Key findings:

- Pre-trained embeddings crucial (+5%)
- Fine-tuning helps slightly
- Simple architecture, strong performance

Regularization Techniques

To prevent overfitting:

1. **Dropout** ($p = 0.5$):
   - Applied before fully connected layer
   - Randomly drop 50% of features during training
   - Prevents co-adaptation
2. **L2 regularization**:
   - Add $\frac{\lambda}{2}||\mathbf{w}||^2$ to loss
   - Prevents large weights
3. **Max-norm constraint**:
   - Constrain $||\mathbf{w}|| \leq s$
   - If exceeds, rescale: $\mathbf{w} \leftarrow \frac{s}{||\mathbf{w}||}\mathbf{w}$
   - Kim used $s = 3$

Hyperparameters (Kim 2014)

Standard settings:

Parameter	**Value**
Embedding dimension	300 (word2vec)
Filter sizes	3, 4, 5
Filters per size	100
Dropout rate	0.5
L2 constraint	3 (max-norm)
Mini-batch size	50
Optimizer	Adadelta

Note: Simple, few hyperparameters to tune!

Character-Level CNNs

Alternative: Apply CNNs to character sequences

Architecture (Zhang et al. 2015):

- Input: Character one-hot vectors
- Convolution over character sequence
- Multiple layers of conv + pooling
- Works without word embeddings!

Advantages:

- No vocabulary limitation
- Captures morphology (prefixes, suffixes)
- Robust to misspellings
- Language-agnostic

Disadvantages:

- Longer sequences (more computation)
- Generally lower accuracy than word-level

Character-Level Example

Word: "unhappiness"

Character embeddings:

- Each character $\rightarrow$ small embedding (e.g., 15-dim)
- Sequence: [u, n, h, a, p, p, i, n, e, s, s]
- Matrix: 11×15

Convolutional filters:

- Size 3: Detects "un-" (prefix), "-ness" (suffix)
- Size 5: Detects "happi" (stem)
- Size 7: Longer patterns

Max pooling $\rightarrow$ Fixed-size character-level word representation

Useful for morphologically rich languages!

Very Deep CNNs

Question: Can we stack many convolutional layers?

VGG-like architectures for text:

- Many small filters (size 3) stacked
- Receptive field grows: $3 \rightarrow 5 \rightarrow 7 \rightarrow \ldots$
- Deeper = more complex patterns

Challenges:

- Vanishing gradients
- Overfitting on small text datasets
- Computational cost

Solutions:

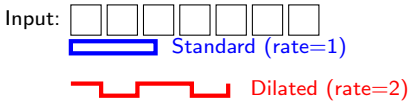
- Residual connections (ResNets)
- Batch normalization
- Large datasets (important!)

Dilated Convolutions

Problem: Standard convolution has limited receptive field

Dilated (Atrous) convolution:

- Skip positions in the input
- Dilation rate r: gaps between filter positions
- Example: $r = 2$ means skip every other position



Benefit: Larger receptive field without more parameters

CNNs vs. RNNs for Text

	CNNs	**RNNs**
Parallelization	Fully parallel	Sequential
Training speed	Fast	Slower
Long-range deps	Limited by filter size	Natural
Fixed-length output	Yes (via pooling)	No
Best for	Classification, short texts	Seq2seq, generation
Parameter sharing	Across positions	Across time

When to use CNNs:

- Sentence/document classification
- When local patterns matter (n-grams)
- Need for speed

When to use RNNs:

- Sequence generation

Modern Use of CNNs in NLP

CNNs are still used in modern NLP:

1. **Hybrid architectures**:
   - CNN for character-level features + RNN for word-level
   - Used in ELMo, some BERT variants
2. **Fast baselines**:
   - Quick experiments before expensive models
   - Production systems needing speed
3. **Specific applications**:
   - Relation extraction (mention-level patterns)
   - Event detection
   - Text classification at scale
4. **Components in larger models**:
   - Gated convolutional networks
   - ConvS2S for machine translation

PyTorch Implementation: CNN Layer

```
import torch
import torch.nn as nn

class TextCNN(nn.Module):
    def __init__(self, vocab_size, embed_dim, num_classes,
                 filter_sizes=[3,4,5], num_filters=100):
        super().__init__()

        # Embedding layer
        self.embedding = nn.Embedding(vocab_size, embed_dim)

        # Convolutional layers (one for each filter size)
        self.convs = nn.ModuleList([
            nn.Conv2d(1, num_filters, (fs, embed_dim))
            for fs in filter_sizes
        ])

        # Fully connected layer
```

PyTorch Implementation: Forward Pass

```
def forward(self, x):
    # x: (batch_size, seq_len)

    # Embedding: (batch_size, seq_len, embed_dim)
    embedded = self.embedding(x)

    # Add channel dimension: (batch_size, 1, seq_len, embed_dim)
    embedded = embedded.unsqueeze(1)

    # Apply convolutions and max-over-time pooling
    conv_outputs = []
    for conv in self.convs:
        # Conv: (batch_size, num_filters, seq_len - fs + 1, 1)
        h = torch.relu(conv(embedded))
        # Max pool: (batch_size, num_filters)
        h = torch.max(h, dim=2)[0].squeeze()
        conv_outputs.append(h)
```

Training Loop

```
model = TextCNN(vocab_size=10000, embed_dim=300,
                num_classes=2)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adadelta(model.parameters())

for epoch in range(num_epochs):
    for batch_x, batch_y in dataloader:
        # Forward pass
        outputs = model(batch_x)
        loss = criterion(outputs, batch_y)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()

        # Gradient clipping (for max-norm constraint)
        torch.nn.utils.clip_grad_norm_(model.parameters(), 3.0)
```

Key Takeaways

CNNs for Text:

- Apply convolution over word embeddings
- Detect local patterns (n-grams, phrases)
- Max-over-time pooling for fixed-size representation

Kim (2014) Architecture:

- Multiple filter sizes (3, 4, 5) $\rightarrow$ different n-gram lengths
- Pre-trained embeddings crucial
- Simple but effective

Advantages:

- Fast (parallelizable)
- Few hyperparameters
- Strong performance on classification

Limitations:

CNNs in the Modern NLP Stack

2014-2016: CNNs popular for text classification

2016-2017: RNNs dominate sequence tasks

2017+: Attention and Transformers take over

But CNNs still relevant:

- Character-level modeling
- Fast baselines
- Hybrid architectures
- Resource-constrained settings

Conceptual importance:

- First successful deep learning for text
- Showed embeddings + simple architecture works
- Paved way for more complex models
- Understanding CNNs helps understand Transformers

Looking Ahead

Next class: RNNs

- Sequential processing
- Hidden states and memory
- LSTMs and GRUs
- Better for variable-length outputs

Connection:

- CNNs: Local patterns, parallel
- RNNs: Sequential dependencies, recurrent
- Both use embeddings
- Both addressed by Transformers (later)

Resources

Key papers:

- Kim (2014): "Convolutional Neural Networks for Sentence Classification"
- Zhang et al. (2015): "Character-level Convolutional Networks for Text Classification"
- Kalchbrenner et al. (2014): "A Convolutional Neural Network for Modelling Sentences"

Implementations:

- PyTorch tutorials on text classification
- AllenNLP library (CNN text classifier)
- Keras/TensorFlow examples

Further reading:

- Goodfellow et al., Deep Learning, Chapter 9 (CNNs)
- Stanford CS224N lecture notes on CNNs

Questions?